Stereoscopic Camera

Summary:

This lesson gives the arm a sense of distance. In Lesson 9, a bounding box told the arm which way to turn, but not how far to reach. Two cameras a fixed distance apart see the same object in slightly different places, and that shift tells us how far away the object is. On Day 1, students define disparity, use $Z = f \cdot B \div d$ to turn it into depth, and find out why depth gets less accurate as objects get farther away. On Day 2, students calibrate a stereo camera with a checkerboard, learn what rectification and a disparity map are, and turn a pixel and its depth into an (x, y, z) point in the arm's base frame. By the end of the lesson, students can calculate the distance to an object from a stereo pair, judge a calibration from its error, and say whether a target point is within the arm's 500 mm reach. That point is the target that inverse kinematics from Lessons 6 to 8 turns into joint angles.

Objective:

By the end of this lesson, students will be able to:

- Explain why one camera cannot measure distance, and why two views of the same scene can
- Measure the disparity of a point from its pixel columns in the left and right images
- Use $Z = f \cdot B \div d$ to find depth, and rearrange it to find the disparity expected at a given depth
- Explain how the baseline and the distance to the object change the accuracy of the depth
- Describe what calibration finds (intrinsics, distortion, and the rotation and translation between the two cameras) and capture a checkerboard set to find it
- Judge a calibration from its reprojection error and its measured baseline
- Explain what rectification does, read a disparity map and find the closest depth it can measure
- Turn a pixel and its depth into a 3D point in the camera frame, and then into the arm's base frame

Standards:

NGSS

- Engineering Design: HS-ETS1-2 (break a complex problem into smaller problems), HS-ETS1-3 (evaluate solutions against trade-offs), HS-ETS1-4 (use a computer model to test a solution)
- Science & Engineering Practices: Developing and Using Models; Planning and Carrying Out Investigations; Using Mathematics and Computational Thinking; Analyzing and Interpreting Data
- Crosscutting Concepts: Systems and System Models; Scale, Proportion, and Quantity

Vocabulary:

Stereo vision: Finding depth by comparing two images of the same scene taken from two places.

Parallax: The apparent shift of an object when you look at it from a different position. Near objects shift more than far ones.

Baseline (B): The distance between the centers of the two cameras, in mm.

Disparity (d): The difference in the horizontal pixel position of the same point in the left and right images, d = x_L − x_R, in pixels.

Depth (Z): How far a point is in front of the camera, measured along the direction the camera looks, in mm.

Focal length (f): A number that describes how strongly the lens zooms. In this lesson it is measured in pixels, as in Lesson 9.

Principal point (cx, cy): The pixel where the camera's forward axis meets the image. It is close to the middle of the image.

Camera frame: The camera's own axes: X to the right, Y downward and Z forward, out of the lens.

Intrinsic parameters: The numbers that belong to one camera: fx, fy, cx and cy, stored in the matrix K, plus the lens distortion.

Lens distortion: The bending of straight lines near the edges of the image, caused by the lens.

Extrinsic parameters: The rotation R and translation T that say where one camera is relative to the other. The length of T is the baseline.

Camera calibration: Finding the intrinsic and extrinsic parameters by photographing a pattern whose size is known.

Checkerboard: A printed grid of squares of known size. The inner corners, where four squares meet, are the points we detect.

Reprojection error: The average distance, in pixels, between a detected corner and where the calibrated camera model says it should be. Smaller is better.

Rectification: Warping both images so that a point appears on the same row in each. After this, d is only a horizontal shift.

Disparity map: An image in which every pixel holds the disparity found for that pixel. Bright pixels are near and dark pixels are far.

max_disp: The largest disparity the matcher searches. Nothing closer than Z_min = f · B ÷ max_disp can be measured.

Stereo matching: Searching along a row for the patch in the right image that looks like a patch in the left image.

Resources & Materials:

- Lesson 10 slideshow, Days 1 and 2 (26 slides; the slide numbers in the Procedures are from this slideshow)
- The arm's stereo camera (RealSense D435i, baseline 50 mm), or two identical webcams fixed rigidly side by side about 60 mm apart. The cameras must not move relative to each other after calibration, so tape or screw them to one bar

- A checkerboard on a flat, rigid board (foam board or a clipboard): 10 × 7 squares, which gives 9 × 6 inner corners. Print at 100% scale, then measure a square with a ruler (this lesson assumes 25 mm)
- Tape marks on a table at 300 mm, 600 mm and 1200 mm from the camera, and an object to place on each mark
- A set of stereo pairs of that object at each mark, taken before class as a backup if the live camera fails
- Computers with Python, the numpy and opencv-python packages, the helper file stereo_helper.py (code in the Appendix), and detect_helper.py from Lesson 9
- Rulers, tape measures and calculators

Teacher note: the helper file and the scripts in the Appendix were tested on rendered images. On 16 rendered checkerboard pairs, the calibration script printed reprojection errors of 0.24, 0.24 and 0.25 px, a focal length of 700.9 px and a baseline of 59.8 mm (true values 700 px and 60 mm). On a rendered flat target 300 mm away, the Day 2 script printed a depth of 301 mm and a distance of 428 mm from the base, which is reachable. They have not been run on the real camera, so do one full calibration before class. The slide examples use B = 60 mm. If you use the RealSense camera, change 60 to 50 in the Day 1 code.

Procedures:

Day 1: Depth from Disparity

- **Opening (6 min)** — Show the title slide and "One Camera Cannot Measure Distance" (slides 1–2). In Lesson 9, one detection told the arm which way to turn, but not how far to reach. A small cup close to the camera and a large cup far away can fill the same pixels. Do the finger test from the slide: hold a finger at arm's length, close one eye, then swap eyes. The finger jumps, but a far wall barely moves. Ask: why? Take two or three ideas before moving on.
- **Two cameras, two views (5 min)** — Show "Two Cameras, Two Views" (slide 3). The finger jumps because each eye sees it from a different place (parallax). A stereo camera is two identical cameras side by side, at the same height and looking the same way, a fixed distance B apart. That distance is the baseline. A nearby point lands at different x positions in the two images, but its y position stays the same.
- **Disparity (7 min)** — Show "Disparity: How Far the Point Moved" (slide 4). Disparity is the horizontal shift of one point between the two images, d = x_L − x_R, in pixels. It is always positive for a point in front of the cameras. A big d means near and a small d means far. Students read the table: the near cup has d = 420 − 350 = 70 px and the far box has d = 410 − 390 = 20 px. Ask: which one is closer, and how do you know?
- **Depth from disparity (7 min)** — Show "Depth from Disparity" (slide 5): Z = f · B ÷ d, where f and d are in pixels and B and Z are in mm, and the same formula solved for disparity, d = f · B ÷ Z. The pixels cancel, so Z comes out in the units of B. Sanity check: disparity doubles when depth halves. Common mistake: mixing units, for example B in cm, which gives Z in cm. Then work "Depth Example" (slide 6): f = 700 px, B = 60 mm, x_L = 420 and x_R = 350. d = 70 px and Z = 700 × 60 ÷ 70 = 42,000 ÷ 70 = 600 mm.
- **Try It Yourself #1 (5 min)** — Students use f = 800 px, B = 50 mm, x_L = 500 and x_R = 420 (slide 7). Reveal the solution: d = 80 px and Z = 800 × 50 ÷ 80 = 500 mm.

- **Try It Yourself #2 (4 min)** — Going the other way: with f = 800 px and B = 50 mm, an object sits at Z = 250 mm. What disparity should the matcher find (slide 8)? Reveal the solution: d = 800 × 50 ÷ 250 = 40,000 ÷ 250 = 160 px.
- **One pixel matters (7 min)** — Show "Far Away Means Less Accurate" (slide 9). Matching is only good to about 1 pixel. With f = 700 px and B = 60 mm, a 1 px mistake gives an error of about 6 mm at 500 mm, 24 mm at 1000 mm and 100 mm at 2000 mm. Error grows roughly with the square of the distance: doubling the distance makes the same 1 px mistake about four times worse, because d is small far away.
- **Choosing the baseline (4 min)** — Show "Choosing the Baseline" (slide 10). A larger B gives more disparity, so a 1 px mistake matters less far away, but the two views overlap less and very close objects can be lost from one view. A smaller B sees close objects well but gets noisy far away. The arm works within 500 mm, so a baseline of a few centimeters is enough. The arm's RealSense D435i camera has B = 50 mm.
- **Check it with code (12 min)** — Show "Check It with Code" (slide 11). f comes from the Lesson 9 formula, f = (W ÷ 2) ÷ tan(HFOV ÷ 2), so `focal_px(640, 60)` gives about 554 px. For x_L = 412 and x_R = 298, d = 114 px and the code prints 292 mm. In pairs, students place the object on the 300 mm, 600 mm and 1200 mm tape marks, read the column of its center in the left and right images with an image viewer, run the code and compare each answer with the tape. Results will be rough because f is only a data sheet estimate until Day 2, and the error should grow at 1200 mm. Ask students to explain why.
- **Recap (3 min)** — Show "Day 1 Recap" (slide 12): disparity is the pixel shift, Z = f · B ÷ d, the error grows with Z^2, and the baseline trades accuracy against overlap. Preview Day 2: where do f and the other camera numbers come from? We calibrate the cameras, and then turn a pixel and a depth into a point the arm can reach.

Day 2: Calibration and 3D Points

- **Recap (3 min)** — Show "Calibration and 3D Points" (slide 13). Quick-fire questions: Does a bigger disparity mean nearer or farther? (Nearer.) What does a longer baseline do to the disparity of a far object? (Makes it bigger.) Why is a far object's depth less accurate? (Its disparity is small, so 1 px is a large part of it.) Day 1 assumed we knew f and that the images were perfectly aligned. Today we measure them.
- **What calibration finds (5 min)** — Show "What Calibration Finds" (slide 14). For each camera, calibration finds the intrinsics K (the focal lengths fx and fy and the center (cx, cy), in pixels) and the lens distortion, which we use to straighten the image. For the pair, it finds the rotation R and translation T from the left camera to the right one. The length of T is the baseline B.
- **Lab: capture the checkerboard (8 min)** — Show "The Checkerboard" (slide 15). The board has 10 × 7 squares of 25 mm, so 9 × 6 inner corners. The software knows where each corner really is on the board and sees where it lands in the image. In pairs, students capture 15 to 20 pairs of images with the whole board visible in both cameras at once. Tilt it, move it near and far, and cover the whole frame. Save them in a folder called calib as left_00.png, right_00.png and so on. Common mistakes: a bent board, a blurry image, a board that is always in the middle of the image, and moving the cameras between captures.
- **Run the calibration (7 min)** — Run the calibration script in the Appendix, then show "Reprojection Error" (slide 16). The reprojection error is the average distance in pixels between

where a corner was found and where the calibrated model says it should be. Under 0.5 px is good; over 1 px means redo it. Students compare the printed baseline with a ruler and expect agreement within a millimeter or two. Common causes of a poor result: the board not flat, motion blur, the board at only one angle or partly out of frame, a wrong square size, or cameras that moved.

- **Rectification (4 min)** — Show "Making the Rows Line Up" (slide 17). Our formula assumes a matching point is on the same row in both images. Before rectification, a point may sit on row 200 in the left image and row 207 in the right. Rectification straightens and aligns both images so the match is on the same row and the search is fast. Watch out: rectifying changes the focal length and center, so always use the new f, cx and cy that `rectify_pair` returns.
- **Disparity map (5 min)** — Show "A Disparity for Every Pixel" (slide 18). StereoSGBM compares small patches along each row and gives a disparity for every pixel. Bright pixels are near. Expect holes where there is no texture (a blank wall) or where one camera sees a spot the other cannot. The matcher only searches up to max_disp, which must be a multiple of 16, so nothing closer than Z_min = f · B ÷ max_disp is seen. With f = 700 px, B = 60 mm and max_disp = 160, Z_min = 700 × 60 ÷ 160 ≈ 263 mm, which covers the arm's 300 to 500 mm range.
- **Try It Yourself #3 (4 min)** — With f = 700 px, B = 60 mm and max_disp = 64, what is the closest depth that can be measured (slide 19)? Reveal the solution: Z_min = 42,000 ÷ 64 ≈ 656 mm. The arm's whole working range would be lost, so use 160 to reach down to about 263 mm.
- **Pixel to 3D point (5 min)** — Show "A Pixel Becomes a 3D Point" (slide 20). With depth Z known, a pixel (u, v) is the camera-frame point X = (u − cx) · Z ÷ f, Y = (v − cy) · Z ÷ f and Z. The pixel origin is the top-left corner, as in Lesson 9, so Y points down. Work "Pixel to Camera Frame" (slide 21): f = 700 px, (cx, cy) = (320, 240), pixel (400, 300) and Z = 600 mm give X = 80 × 600 ÷ 700 = 68.6 mm and Y = 60 × 600 ÷ 700 = 51.4 mm, so the point is (68.6, 51.4, 600) mm.
- **Try It Yourself #4 (4 min)** — Students use f = 800 px, (cx, cy) = (320, 240), pixel (240, 300) and Z = 400 mm (slide 22). Reveal the solution: X = −80 × 400 ÷ 800 = −40 mm and Y = 60 × 400 ÷ 800 = 30 mm, so the point is (−40, 30, 400) mm. Negative X means left of the camera's center line.
- **Camera frame to base frame (4 min)** — Show "Camera Frame to Base Frame" (slide 23). The camera sits at (cam_x, cam_y, cam_z) in the base frame and looks along the arm's +x direction, so x_b = cam_x + Z, y_b = cam_y − X and z_b = cam_z − Y. The signs flip because camera X points right but the arm's +y points left, and camera Y points down but +z points up. Then check reach: the arm has two 250 mm links, so a target farther than 500 mm from the shoulder is out of reach. Common mistake: forgetting that image y points down, which flips the sign of z_b.
- **Where should the gripper go? (4 min)** — Work "Where Should the Gripper Go?" (slide 24). The camera is at (100, 0, 150) mm and the cup's camera-frame point is (34.3, 25.7, 300) mm. Then x_b = 400 mm, y_b = −34.3 mm and z_b = 124.3 mm. The distance from the base is $\sqrt{(400^2 + 34.3^2 + 124.3^2)}$ = 420.3 mm, so it is within reach, and θ_B = atan2(−34.3, 400) = −4.9°, as in Lesson 6. Ask: from the camera, the cup is atan(34.3 ÷ 300) = 6.5° to the right, so why does the base only turn 4.9°? (The base sits 100 mm behind the camera.) This is the exact angle that Lesson 9 could only estimate.
- **Check it with code (5 min)** — Show "The Whole Workflow in Code" (slide 25) and run the Day 2 script in the Appendix. It rectifies a stereo pair, makes a disparity map, takes the target box from the Lesson 9 detector (run on the rectified left image), reads the median disparity inside it, and turns the box center and that depth into a point in the base frame. It then prints the distance

from the base and whether it is inside the 500 mm reach. Students check the helper functions with the worked examples: (68.6, 51.4, 600) and (400, −34.3, 124.3). Always print the reprojection error first, and do not trust the depth if it is above 1 px.

- **Wrap-up (2 min)** — Show "What We Can Now Do" (slide 26): turn disparity into depth and judge its accuracy; choose a baseline; calibrate a stereo pair and check the reprojection error; rectify, build a disparity map and turn a pixel into arm coordinates. Detection tells the arm what and which way. Stereo tells it how far. Together they give a 3D target, and inverse kinematics from Lessons 6 to 8 turns it into joint angles.

Assessment:

Students complete the Try It Yourself problems (#1–#4) and check each answer against the solution on the slide. They can check #4 by calling `pixel_to_camera` from the helper file.

Use these questions as an exit ticket or short quiz. The answers are in red.

- Why does a nearby object shift more between the left and right images than a far one?
  Answer: Each camera sees the object from a different direction (parallax), and the nearer the object is, the bigger that difference in direction is.
- A point is at column 412 in the left image and column 398 in the right. What is its disparity?
  Answer: d = 412 − 398 = 14 px.
- With f = 700 px and B = 60 mm, find the depth for d = 14 px.
  Answer: Z = 700 × 60 ÷ 14 = 3,000 mm.
- With f = 700 px and B = 60 mm, an object is 450 mm away. What disparity do we expect?
  Answer: d = 700 × 60 ÷ 450 = 93.3 px.
- Why is depth less accurate for far objects? Use numbers.
  Answer: A far object has a small disparity, so a 1 px error is a large part of it. With f = 700 px and B = 60 mm, a 1 px error at 500 mm gives about 6 mm of depth error, but at 2000 mm it gives about 100 mm.
- What does a longer baseline do for the accuracy of far objects, and what does it cost?
  Answer: It makes the disparity bigger for the same distance, so a 1 px error matters less. The cost is that the two cameras share less of the scene up close, so near objects may be seen by only one camera.
- Find the focal length in pixels for an image 1280 px wide with a 90° horizontal field of view.
  Answer: f = 640 ÷ tan 45° = 640 px.
- With f = 700 px and (cx, cy) = (640, 360), find the camera-frame point for the pixel (740, 320) at Z = 300 mm.
  Answer: X = 100 × 300 ÷ 700 = 42.9 mm and Y = −40 × 300 ÷ 700 = −17.1 mm, so the point is (42.9, −17.1, 300) mm.
- Name three things that calibration finds.

Answer: Any three of: the focal length and center of each camera (K), the lens distortion, and the rotation and translation between the two cameras (whose length is the baseline).

- A calibration gives a reprojection error of 2.4 px. What should you do?
  Answer: Do not use it. Recapture the images, checking that the board is flat and sharp, that it appears in all parts of the image at different tilts, that the square size is right and that the cameras did not move.

- What does rectification do, and why does it help?
  Answer: It warps both images so that a point is on the same row in each. Then the matcher only has to search along one row, and the disparity is just a horizontal shift.

- With f = 700 px and B = 60 mm, the arm must see objects 300 mm away. Is max_disp = 128 enough?
  Answer: No. Z_min = 42,000 ÷ 128 ≈ 328 mm, which is farther than 300 mm. An object at 300 mm has d = 140 px, so use max_disp = 144 or 160.

- The camera is at (100, 0, 150) mm and a point in the camera frame is (−40, 30, 400) mm. Find the point in the base frame. Can the arm reach it?
  Answer: (100 + 400, 0 − (−40), 150 − 30) = (500, 40, 120) mm. Its distance from the base is $\sqrt{(500^2 + 40^2 + 120^2)}$ = 515.8 mm, more than 500 mm, so it is out of reach.

Appendix: stereo_helper.py

Save this code as stereo_helper.py in the same folder as the student code. Install the packages first with `pip install numpy opencv-python`.

```
"""Helper functions for the Stereoscopic Camera lesson.
Pixel coordinates: (0, 0) is the TOP-LEFT of the image, x to the right, y down.
Lengths are in mm. Focal length f is measured in PIXELS.
Camera frame: x right, y down, z forward (out of the lens).
"""
import math
import numpy as np


# ---------- Day 1: disparity and depth ----------

def focal_px(image_width, hfov_deg):
    """Focal length in pixels from the image width and horizontal FOV."""
    return (image_width / 2) / math.tan(math.radians(hfov_deg) / 2)


def depth_from_disparity(d, f, baseline):
    """Z = f * B / d.  d and f in pixels, baseline and Z in mm."""
    return f * baseline / d


def disparity_from_depth(Z, f, baseline):
    """d = f * B / Z."""
    return f * baseline / Z
```

```
def pixel_to_camera(u, v, Z, f, cx, cy):
    """Pixel (u, v) at depth Z -> 3D point (X, Y, Z) in the camera frame."""
    return (u - cx) * Z / f, (v - cy) * Z / f, Z


def camera_to_base(p_cam, cam_pos):
    """Camera looks along the base +x axis, mounted at cam_pos = (x, y, z).
    Camera frame: X right, Y down, Z forward.
    Base frame: x forward, y left, z up."""
    X, Y, Z = p_cam
    return cam_pos[0] + Z, cam_pos[1] - X, cam_pos[2] - Y


# ---------- Day 2: calibration and disparity maps ----------

def board_points(pattern=(9, 6), square=25.0):
    """3D corner positions of a checkerboard lying flat (z = 0), in mm.
    pattern = number of INNER corners (columns, rows)."""
    pts = np.zeros((pattern[0] * pattern[1], 3), np.float32)
    pts[:, :2] = np.mgrid[0:pattern[0], 0:pattern[1]].T.reshape(-1, 2) * square
    return pts


def find_corners(gray, pattern=(9, 6)):
    """Find the checkerboard corners in a grayscale image (None if none)."""
    import cv2
    ok, corners = cv2.findChessboardCorners(gray, pattern)
    if not ok:
        return None
    crit = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
    return cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), crit)


def calibrate_camera(obj_points, img_points, image_size):
    """One camera. Returns (rms_error_px, K, dist).
    K = [[fx, 0, cx], [0, fy, cy], [0, 0, 1]] is the intrinsic matrix."""
    import cv2
    rms, K, dist, _, _ = cv2.calibrateCamera(obj_points, img_points,
                                             image_size, None, None)
    return rms, K, dist


def calibrate_stereo(obj_points, left_pts, right_pts, K1, d1, K2, d2,
                     image_size):
    """Two cameras. Returns (rms_error_px, R, T): the rotation and translation
    from the left camera to the right camera. |T| is the baseline."""
    import cv2
    flags = cv2.CALIB_FIX_INTRINSIC
    rms, _, _, _, _, R, T, _, _ = cv2.stereoCalibrate(
        obj_points, left_pts, right_pts, K1, d1, K2, d2, image_size,
        flags=flags)
    return rms, R, T


def baseline_mm(T):
    return float(np.linalg.norm(T))
```

```
def disparity_map(left_gray, right_gray, max_disp=160, block=5):
    """Disparity in pixels for every pixel (images must be rectified).
    max_disp must be a multiple of 16. Nothing closer than f * B / max_disp
    is seen. With f = 700 px and B = 60 mm, 160 reaches down to 263 mm."""
    import cv2
    sgbm = cv2.StereoSGBM_create(minDisparity=0, numDisparities=max_disp,
                                 blockSize=block,
                                 P1=8 * block * block, P2=32 * block * block,
                                 uniquenessRatio=10)
    return sgbm.compute(left_gray, right_gray).astype(np.float32) / 16.0


def rectify_pair(left, right, K1, d1, K2, d2, R, T, image_size):
    """Warp both images so matching points lie on the same row.
    Returns (left_rect, right_rect, f, cx, cy): the focal length and principal
    point (px) of the rectified cameras. Use THESE in Z = f * B / d, not the
    old values."""
    import cv2
    R1, R2, P1, P2, Q, _, _ = cv2.stereoRectify(K1, d1, K2, d2, image_size,
                                                R, T, alpha=0)
    m1 = cv2.initUndistortRectifyMap(K1, d1, R1, P1, image_size, cv2.CV_32FC1)
    m2 = cv2.initUndistortRectifyMap(K2, d2, R2, P2, image_size, cv2.CV_32FC1)
    left_rect = cv2.remap(left, m1[0], m1[1], cv2.INTER_LINEAR)
    right_rect = cv2.remap(right, m2[0], m2[1], cv2.INTER_LINEAR)
    f, cx, cy = float(P1[0, 0]), float(P1[0, 2]), float(P1[1, 2])
    return left_rect, right_rect, f, cx, cy


def median_disparity(disp, box):
    """Median disparity inside a box (x_min, y_min, x_max, y_max), ignoring
    pixels where no match was found. Returns None if nothing matched."""
    x1, y1, x2, y2 = [int(v) for v in box]
    patch = disp[y1:y2, x1:x2]
    good = patch[patch > 0]
    return float(np.median(good)) if good.size else None


if __name__ == "__main__":
    f = 700
    print(round(depth_from_disparity(14, f, 60)))  # expected: 3000
```

Day 1 “Check It with Code” script. Change x_L and x_R to the columns read from your images. It prints 292 mm for the slide’s numbers.

```
from stereo_helper import focal_px, depth_from_disparity

f = focal_px(640, 60)          # Lesson 9: about 554 px
x_L, x_R = 412, 298            # read from the two images
Z = depth_from_disparity(x_L - x_R, f, 60)
print(round(Z), "mm")          # 292
```

Day 2 calibration script. Put the captured pairs in a folder called calib as left_00.png, right_00.png, left_01.png and so on. It saves the results in calibration.npz for the next script.

```
import glob, numpy as np, cv2
from stereo_helper import *

PATTERN, SQUARE = (9, 6), 25.0         # inner corners, square size in mm
left_files = sorted(glob.glob("calib/left_*.png"))
right_files = sorted(glob.glob("calib/right_*.png"))
```

```
obj_pts, left_pts, right_pts = [], [], []
for lf, rf in zip(left_files, right_files):
    gl = cv2.imread(lf, cv2.IMREAD_GRAYSCALE)
    gr = cv2.imread(rf, cv2.IMREAD_GRAYSCALE)
    cl, cr = find_corners(gl, PATTERN), find_corners(gr, PATTERN)
    if cl is None or cr is None:
        print("skipped", lf)
        continue
    obj_pts.append(board_points(PATTERN, SQUARE))
    left_pts.append(cl)
    right_pts.append(cr)
size = (gl.shape[1], gl.shape[0])

rms1, K1, d1 = calibrate_camera(obj_pts, left_pts, size)
rms2, K2, d2 = calibrate_camera(obj_pts, right_pts, size)
rmsS, R, T = calibrate_stereo(obj_pts, left_pts, right_pts,
                              K1, d1, K2, d2, size)
print(f"pairs used: {len(obj_pts)}")
print(f"reprojection error (px): left {rms1:.2f}, right {rms2:.2f}, "
      f"stereo {rmsS:.2f}")
print(f"focal length (px): {K1[0, 0]:.1f}   "
      f"baseline (mm): {baseline_mm(T):.1f}")
np.savez("calibration.npz", K1=K1, d1=d1, K2=K2, d2=d2, R=R, T=T, size=size)
```

Day 2 “Check It with Code” script. Run the Lesson 9 detector on the rectified left image to get the box, and set CAM_POS to where the camera sits on your arm. Students should place their targets between about 300 mm and 450 mm from the camera.

```
import math, numpy as np, cv2
from stereo_helper import *

cal = np.load("calibration.npz")
K1, d1, K2, d2 = cal["K1"], cal["d1"], cal["K2"], cal["d2"]
R, T = cal["R"], cal["T"]
size = tuple(int(v) for v in cal["size"])
CAM_POS = (100, 0, 150)       # where the camera sits in the base frame (mm)

left = cv2.imread("scene_left.png", cv2.IMREAD_GRAYSCALE)
right = cv2.imread("scene_right.png", cv2.IMREAD_GRAYSCALE)
left_r, right_r, f, cx, cy = rectify_pair(left, right, K1, d1, K2, d2,
                                          R, T, size)
B = baseline_mm(T)

disp = disparity_map(left_r, right_r, max_disp=160)
box = (500, 300, 700, 420)    # target box from the Lesson 9 detector,
                              # run on the rectified left image left_r
d = median_disparity(disp, box)
if d is None:
    print("No match inside the box")
else:
    Z = depth_from_disparity(d, f, B)
    u, v = (box[0] + box[2]) / 2, (box[1] + box[3]) / 2
    p_base = camera_to_base(pixel_to_camera(u, v, Z, f, cx, cy), CAM_POS)
    dist = math.sqrt(sum(c * c for c in p_base))
    print(f"depth: {Z:.0f} mm")
    print("target in base frame (mm):", [round(c) for c in p_base])
    print(f"distance from base: {dist:.0f} mm ->",
          "reachable" if dist <= 500 else "too far")
```